

Micro-Projet : Réseau de neurones

PROCACCIA / LEPOURTOIS



École Centrale Nantes SEC3

Table des matières

1. OBJECTIFS	3
2. ANALYSE DU CODE SEQUENTIELLE	4
2.1. Structure du code et analyse des performances	4
2.2. Structure fonctionnelle	4
2.3. Commandes de compilation et profilage	4
2.3.1. Compilation avec profilage	5
2.3.2. Exécution du programme	5
2.3.3. Analyse des performances	5
2.4. Objectif de cette analyse	5
2.5. Interprétation des résultats du profilage	6
2.5.1. Résultat du profilage	6
2.5.2. Visualisation graphique	7
2.5.3. Conséquences pour l'optimisation	7
3. OPTIMISATION SUR GPU	8
3.1. GPU classique	8
3.2. Débridage JETSON	9
3.3. GPU Mémoire unifiée	9
3.4. Portage des autres fonctions matricielles sur GPU et optimisation de <code>matrix_dot</code> avec mémoire partagée et tuiles	10
3.5. Optimisation mémoire de forward et backward	11
3.6. Réduction des <code>cudaDeviceSynchronize()</code>	11
3.7. Fusion des kernels pour limiter les <code>cudaLaunchKernel()</code>	11

1. Objectifs

Mettre en pratique les concepts abordés en cours et en travaux dirigés sur le parallélisme des données et la programmation en CUDA.

L'objectif est de démontrer notre maîtrise de la programmation sur GPU ainsi que notre capacité à concevoir des traitements adaptés aux données parallèles.

Il faut programmer un réseau de neurones artificiels à partir de zéro. Un code séquentiel en C nous est fourni comme base de travail. Notre objectif sera d'optimiser la vitesse de calcul sur GPU

Pour cela, nous devons :

- Identifier les parties de l'application les plus consommatrices en temps en effectuant des mesures sur le code fourni.
- Déterminer une ou plusieurs approches exploitant le parallélisme.
- Implémenter ces approches sous différentes variantes.
- Mesurer les performances des implémentations en fonction de divers paramètres.
- Analyser les résultats obtenus et les mettre en lien avec les concepts vus en cours.

Ce rapport a pour but d'éclairer notre démarche et notre raisonnement. Nous présenterons donc notre méthode de penser, nos résultats et nos constats. Nous expliquerons le fonctionnement ou non de certaines parties et ce que nous avons mis en place pour corriger les éventuelles erreurs.

2. Analyse du code séquentielle

2.1. Structure du code et analyse des performances

Le code fourni implémente un réseau de neurones artificiels de type perceptron multicouche (MLP), en C, sans appel à de bibliothèques externes. Il s'appuie sur des structures de matrices (**'matrix_t'**) pour représenter les poids, les biais et les activations des différentes couches.

L'entraînement du réseau repose sur l'algorithme classique de descente de gradient avec rétropropagation, appliqué à des **mini-batches** extraits du jeu de données **MNIST**.

2.2. Structure fonctionnelle

Voici les fichiers principaux :

- **'main.c'** : boucle d'entraînement, initialisation du réseau, mesure de la précision.
- **'ann.c / ann.h'** : structures **'ann_t'** et **'layer_t'**, fonctions **'forward()'** et **'backward()'**.
- **'matrix.c / matrix.h'** : opérations de base sur matrices (produit, somme, transpose, etc.).
- **'mnist.c / mnist.h'** : lecture des fichiers binaires MNIST.

Le réseau est composé de trois couches :

- Une couche d'entrée de 784 neurones (28×28 pixels),
- Une couche cachée de 30 neurones,
- Une couche de sortie de 10 neurones (représentant les chiffres de 0 à 9).

2.3. Commandes de compilation et profilage

Pour analyser les performances du programme et identifier les fonctions les plus coûteuses, nous avons utilisé l'outil GNU **'gprof'**.

2.3.1. Compilation avec profilage

« `gcc -pg ann.c main.c matrix.c mnist.c -lm` »

L'option '**-pg**' active la génération du fichier '**gmon.out**' nécessaire pour '**gprof**'.

2.3.2. Exécution du programme

« `./a.out` »

Cela lance l'apprentissage du réseau de neurones. À la fin de l'exécution, le fichier '**gmon.out**' est généré automatiquement.

2.3.3. Analyse des performances

« `gprof -Q -b a.out` »

Cette commande lit '**gmon.out**' et affiche un rapport de profil plat, listant le **temps passé dans chaque fonction**, le **nombre d'appels** et le **temps cumulé**. L'option '**-b**' supprime les descriptions verbales et '**-Q**' évite les symboles inutiles.

2.4. Objectif de cette analyse

Cette analyse a pour but **d'identifier les fonctions à cibler en priorité** pour la mise en œuvre d'une optimisation GPU. Les fonctions consommatrices en temps seront réécrites en CUDA ou remplacées par des appels à des bibliothèques accélérées (comme cuBLAS), afin de réduire significativement le temps d'entraînement.

2.5. Interprétation des résultats du profilage

Afin d'identifier les fonctions critiques du code séquentiel, nous avons utilisé 'gprof' pour générer un profil d'exécution. Ce profil permet de déterminer avec précision quelles fonctions consomment le plus de temps CPU durant l'apprentissage du réseau de neurones. Voici ce qu'on obtient :

```
● pehladik@pehladik-desktop:~/Documents/CPU_2_GPU_neural_network_MNIST$ gprof -Q -b a.out  
Flat profile:
```

Each sample counts as 0.01 seconds.

% time	cumulative seconds	self seconds	calls	self s/call	total s/call	name
91.52	54.68	54.68	38733	0.00	0.00	matrix_dot
2.40	56.11	1.43	18745	0.00	0.00	matrix_minus
2.04	57.34	1.22	14996	0.00	0.00	matrix_scalar
1.91	58.48	1.14	11247	0.00	0.00	matrix_transpose
1.83	59.57	1.09	4997	0.00	0.00	populate_minibatch
0.10	59.63	0.06	9994	0.00	0.00	matrix_sum
0.08	59.68	0.05	17492	0.00	0.00	matrix_function
0.08	59.73	0.05	7498	0.00	0.00	hadamard_product
0.05	59.76	0.03	7996800	0.00	0.00	sigmoid
0.05	59.79	0.03	2399360	0.00	0.00	dsigmoid
0.02	59.80	0.01	4997	0.00	0.01	forward
0.00	59.80	0.00	74986	0.00	0.00	alloc_matrix
0.00	59.80	0.00	74971	0.00	0.00	destroy_matrix
0.00	59.80	0.00	23820	0.00	0.00	normalRand
0.00	59.80	0.00	3749	0.00	0.01	backward
0.00	59.80	0.00	4	0.00	0.00	make_uint32
0.00	59.80	0.00	3	0.00	0.00	create_layer
0.00	59.80	0.00	3	0.00	0.00	zero_to_n
0.00	59.80	0.00	2	0.00	3.68	accuracy
0.00	59.80	0.00	2	0.00	0.00	init_weight
0.00	59.80	0.00	2	0.00	0.00	read_images
0.00	59.80	0.00	2	0.00	0.00	read_labels
0.00	59.80	0.00	1	0.00	0.00	create_ann
0.00	59.80	0.00	1	0.00	0.00	shuffle

2.5.1. Résultat du profilage

L'analyse révèle que la fonction '**matrix_dot**', responsable du produit matriciel, domine très largement le temps d'exécution :

- Elle représente **plus de 91 % du temps total** (soit 54 secondes sur 60),
- Elle est appelée à **chaque étape de la propagation** avant ('forward') et de la rétropropagation ('backward'), ce qui explique sa fréquence élevée.

D'autres fonctions sont également appelées de manière intensive, bien que leur poids relatif reste modéré :

- '**matrix_minus**' (2.59 %)

- 'matrix_scalar' (2.03 %)
- 'populate_minibatch' (1.87 %)
- 'matrix_transpose' (1.86 %)

Toutes les autres fonctions du code ne représentent qu'environ 0.94 % du temps cumulé.

2.5.2. Visualisation graphique

Le graphique ci-dessous illustre la répartition du temps d'exécution par fonction, en regroupant les fonctions les moins coûteuses sous une seule catégorie nommée "autres". Cela permet de mettre en évidence la disproportion entre '**matrix_dot**' et le reste du programme.

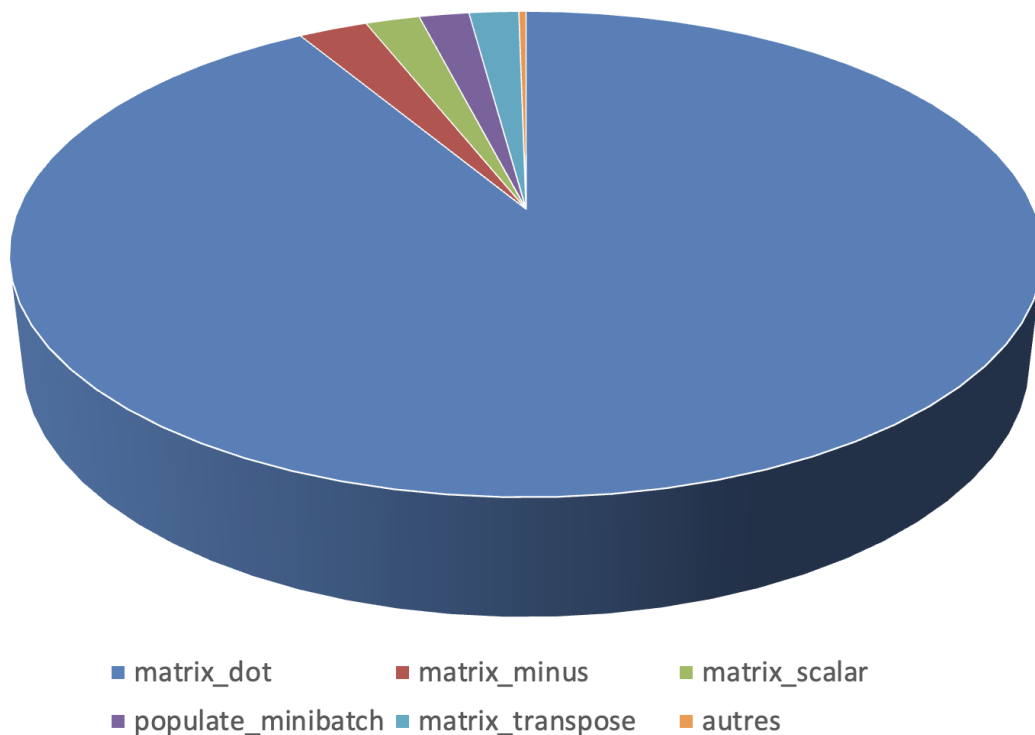


Figure 1 : Répartition du temps d'exécution par fonction ('gprof')

2.5.3. Conséquences pour l'optimisation

Ces résultats justifient de concentrer en priorité l'optimisation GPU sur la fonction '**matrix_dot**', dont l'accélération pourrait à elle seule entraîner une amélioration significative des performances globales du réseau.

Dans un second temps, les fonctions '**matrix_minus**', '**matrix_scalar**', '**matrix_transpose**' et '**populate_minibatch**' peuvent également être considérées comme des cibles pertinentes d'optimisation parallèle. (**Temps total = 60 s**)

3. Optimisation sur GPU

3.1. GPU classique

Dans la version initiale du projet, tous les calculs (notamment les produits matriciels) étaient exécutés sur le **CPU**. L'analyse de profil a montré que la fonction **matrix_dot** représentait à elle seule plus de **90 % du temps d'exécution** total. Afin de tirer parti des capacités massivement parallèles du **GPU**, nous avons réécrit cette fonction sous la forme d'un **kernel CUDA**, permettant une exécution concurrente sur plusieurs milliers de threads.

Le **profil GPU** (Figure 1) montre que **matrix_dot_kernel** occupe désormais **80 % du temps GPU**. Cela confirme que le calcul matriciel a bien été transféré sur le GPU, réduisant considérablement la charge du CPU. Toutefois, cette première approche « classique » repose sur un **enchaînement séquentiel** de transferts mémoire CPU→GPU, calcul GPU, puis transfert mémoire GPU→CPU.

```
==9480== Profiling result:
```

Type	Time(%)	Time	Calls	Avg	Min	Max	Name
GPU activities:	80.10%	12.7667s	38733	329.61us	17.187us	1.3645ms	matrix_dot_kernel(double const *, double const *, double*, int, int, int)
	13.69%	2.18157s	77466	28.161us	2.4470us	190.52us	[CUDA memcpy HtoD]
	6.22%	991.00ms	38733	25.585us	6.8230us	184.53us	[CUDA memcpy DtoH]
API calls:	53.20%	55.5068s	116199	477.69us	36.250us	237.79ms	cudaMemcpy
	33.13%	34.5648s	116199	297.46us	10.677us	462.40ms	cudaMalloc
	9.33%	9.73668s	116199	83.793us	11.823us	320.63ms	cudaFree
	4.34%	4.52344s	38733	116.79us	43.698us	17.427ms	cudaLaunchKernel
	0.00%	126.35us	97	1.3020us	729ns	30.312us	cuDeviceGetAttribute
	0.00%	12.605us	1	12.605us	12.605us	12.605us	cuDeviceTotalMem
	0.00%	6.4580us	3	2.1520us	1.2500us	3.0720us	cuDeviceGetCount
	0.00%	3.8550us	2	1.9270us	1.4590us	2.3960us	cuDeviceGet
	0.00%	1.9270us	1	1.9270us	1.9270us	1.9270us	cuDeviceGetName
	0.00%	938ns	1	938ns	938ns	938ns	cuDeviceGetUuid

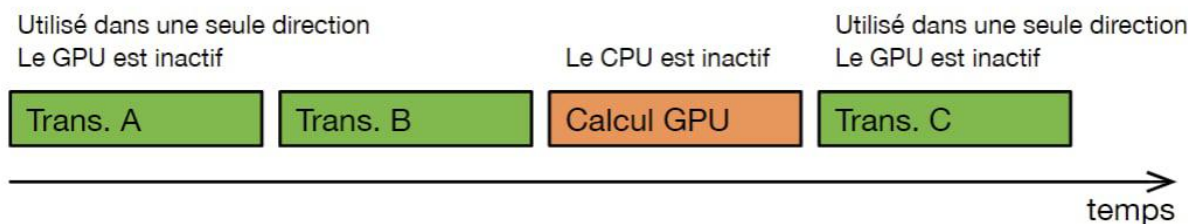
Figure 2 : 1 epoch : 120 secondes

Dans cette configuration, qu'on a réduit considérablement la longueur de temps de la fonction **matrix_dot**. De plus on observe que les transferts de données représentent une part significative du coût global :

Environ **13.6 %** pour **cudaMemcpy HtoD**

Et **6.2 %** pour **cudaMemcpy DtoH**

De plus, les appels API CUDA (notamment **cudaMalloc**, **cudaMemcpy**, etc.) s'accumulent à plus de **50 % du temps d'exécution**, ce qui **limite le gain global** de performance. L'un des problèmes fondamentaux mis en évidence est le **caractère bloquant et séquentiel** de cette approche.



Ce schéma met en évidence que le **CPU et le GPU restent inactifs une partie du temps**, faute de parallélisation entre transferts mémoire et calculs. Cela

suggère qu'il serait possible d'**optimiser davantage l'utilisation du GPU** en mettant en œuvre des **transferts asynchrones et du "streaming"**, afin de **chevaucher les transferts et les calculs**. Pour optimiser en premier temps ces transferts mémoire on va mettre en œuvre la mémoire unifiée.

(Temps total = 120 s)

3.2. Débridage JETSON

Dans la version initiale de notre projet, l'ensemble des opérations, y compris les produits matriciels réalisés par la fonction **matrix_dot**, étaient exécutées sur le CPU. Cette fonction monopolisait plus de 90 % du temps d'exécution, rendant le calcul peu efficace pour un réseau de neurones entraîné sur un grand nombre de données. Afin d'exploiter le parallélisme massif offert par le GPU, nous avons entrepris un portage progressif du code, en commençant par une exécution GPU classique, avec des transferts mémoire explicites de données entre le CPU et le GPU.

Avant de procéder aux mesures de performance, nous avons configuré notre Jetson Nano en mode MAXN afin de maximiser les ressources disponibles du GPU. Cette configuration s'effectue via les commandes **sudo nvpmode -m 0** et **sudo jetson_clocks**, comme illustré dans la figure ci-dessous.

```
pehladi@pehladi-desktop:~$ sudo nvpmode -m 0
pehladi@pehladi-desktop:~$ sudo jetson_clocks
pehladi@pehladi-desktop:~$ sudo nvpmode -q
NVPM WARN: fan mode is not set!
NV Power Mode: MAXN
0
```

L'introduction du kernel CUDA **matrix_dot_kernel** dans cette première version GPU a permis une réduction immédiate du temps CPU, en déléguant le calcul matriciel au GPU. Toutefois, cette approche reposait encore sur des transferts mémoire explicites (**cudaMemcpy**) entre CPU et GPU avant et après chaque opération, introduisant des phases d'inactivité sur l'un ou l'autre processeur. Le schéma classique CPU → transfert mémoire → calcul GPU → transfert mémoire inverse révèle une perte de temps liée à l'alternance séquentielle des tâches, et ce malgré l'accélération du calcul.

3.3. GPU Mémoire unifiée

Pour améliorer davantage les performances et simplifier la gestion mémoire, nous avons adopté l'utilisation de la mémoire unifiée (**cudaMallocManaged**), une fonctionnalité qui permet de partager automatiquement les données entre le CPU et le GPU, sans avoir à déclencher manuellement les transferts mémoire. Le profil obtenu (ci-dessous) montre que les temps liés aux transferts

mémoire sont atténués, et que le kernel GPU occupe désormais 100 % du temps d'exécution GPU.

```
==10155== Profiling application: ./a.out
==10155== Profiling result:
```

Type	Time(%)	Time	Calls	Avg	Min	Max	Name
GPU activities:	100.00%	1.21697s	38733	31.419us	1.7180us	150.10us	matrix_dot_kernel(double const *, double const *, double*, int, int, int)
API calls:	39.56%	13.4012s	38733	345.99us	70.000us	16.523ms	cudaLaunchKernel
	25.25%	8.55467s	74986	114.08us	77.604us	661.16ms	cudaMallocManaged
	21.85%	7.40200s	74971	98.731us	78.594us	9.7599ms	cudaFree
	13.33%	4.51560s	38733	116.58us	49.635us	5.2689ms	cudaDeviceSynchronize
	0.00%	124.53us	97	1.2830us	729ns	29.427us	cuDeviceGetAttribute
	0.00%	11.146us	1	11.146us	11.146us	11.146us	cuDeviceTotalMem
	0.00%	5.9370us	3	1.9790us	1.4580us	2.9170us	cuDeviceGetCount
	0.00%	3.5420us	2	1.7710us	1.3540us	2.1880us	cuDeviceGet
	0.00%	1.8750us	1	1.8750us	1.8750us	1.8750us	cuDeviceGetName
	0.00%	886ns	1	886ns	886ns	886ns	cuDeviceGetUuid

Ce dernier ajustement permet une exécution plus rapide, plus fluide et allège considérablement la complexité du code. Le temps global d'exécution diminue encore, démontrant que la gestion de la mémoire et le modèle d'exécution jouent un rôle crucial dans l'efficacité globale d'un traitement parallèle sur GPU. **(Temps total = 35,09 s 3,4 fois plus rapide que le GPU classique)**

3.4. Portage des autres fonctions matricielles sur GPU et optimisation de matrix_dot avec mémoire partagée et tuiles

Dans un second temps, nous avons porté d'autres fonctions matricielles essentielles sur le GPU, afin de **maximiser le parallélisme** et de **garder les données sur le GPU** :

- matrix_minus_kernel
- matrix_scalar_kernel
- matrix_sum_kernel
- hadamard_product_kernel
- matrix_sigmoid_kernel

Le fait de regrouper un maximum d'opérations sur le GPU a permis de réduire les échanges CPU ↔ GPU et d'améliorer l'efficacité globale du traitement.

Nous avons ensuite optimisé le kernel matrix_dot_kernel en y introduisant l'utilisation de la **mémoire partagée (__shared__)** et une approche par **tuiles (tiling)**.

Cette technique permet de **charger localement en mémoire rapide des sous-matrices**, limitant ainsi les accès à la mémoire globale. Cette optimisation a permis de réduire le temps d'exécution de matrix_dot_kernel et d'améliorer l'utilisation des ressources GPU. **(Temps total = 33 s, 1,06 fois plus rapide que la mémoire unifiée, on gagne du temps mais cela reste relativement négligeable)**

3.5. Optimisation mémoire de forward et backward

Au départ, les fonctions forward et backward réalisaient des **allocations/désallocations répétées** à chaque itération. Nous avons optimisé ce comportement en utilisant des **variables persistantes**, notamment :

- `z1[l]` : résultat de la multiplication des poids `W_l` par les activations `a_{l-1}`
- `z2[l]` : résultat de la multiplication des biais `b_l` par un vecteur de 1

Ces tableaux sont désormais **pré-alloués** en mémoire unifiée et **réutilisés** tout au long de l'exécution, ce qui a permis de **réduire le nombre d'appels coûteux à `cudaMallocManaged` et `cudaFree`**. *(Temps total = 23 s, 1,4 fois plus rapide, qui le portage des autres fonctions matricielles sur le GPU et l'optimisation des tuiles)*

3.6. Réduction des `cudaDeviceSynchronize()`

L'analyse des profils a révélé que certaines fonctions contenaient des synchronisations (`cudaDeviceSynchronize`) superflues, qui ralentissaient l'exécution.

Nous avons supprimé ces synchronisations redondantes dans les fonctions appelant des kernels, et **ajouté des synchronisations uniquement là où cela est nécessaire** (dans les fonctions forward et backward), afin d'assurer la cohérence tout en optimisant les performances. *(Temps total = 9s, 2,5 fois plus rapide que l'optimisation forward et backward)*

3.7. Fusion des kernels pour limiter les `cudaLaunchKernel()`

Enfin, nous avons fusionné plusieurs kernels pour réduire le nombre d'appels à **`cudaLaunchKernel()`**. Cela a permis de **minimiser les coûts liés aux transitions entre kernels** et d'optimiser l'occupation du GPU.

Par exemple :

- `fused_dot_scalar_minus` regroupe produit matriciel, multiplication scalaire et soustraction
- `fused_function_hadamard` combine plusieurs opérations matricielles

Cette fusion a permis d'améliorer encore le **débit d'exécution** et de **stabiliser les temps par époque**. *(Temps total = 7 s, 1,2 fois plus rapide que la réduction des `cudaDeviceSynchronize()`)*

Soit 17 fois plus rapide que le GPU classique !!!

Soit 8,6 fois plus rapide que le CPU classique !!!

4. Conclusion

La démarche d'optimisation que nous avons menée a permis d'obtenir des performances **GARGANTUESQUES** sur notre projet de réseau de neurones MNIST porté sur GPU.

En suivant une approche progressive et itérative, nous avons :

- Commencé par un portage simple en CUDA "classique", avec des transferts mémoire explicites,
- Introduit l'utilisation de la mémoire unifiée (cudaMallocManaged),
- Porté l'ensemble des fonctions matricielles sur le GPU,
- Optimisé les kernels, notamment matrix_dot_kernel, en exploitant la mémoire partagée et le tiling,
- Amélioré la gestion mémoire dans forward et backward grâce à des variables persistantes,
- Supprimé les synchronisations superflues du GPU,
- Et enfin, fusionné plusieurs kernels pour minimiser les appels à l'API call : cudaLaunchKernel().

Ces optimisations nous ont permis de **diviser le temps d'exécution par un facteur 8,6** par rapport à la première version CPU classique.

La version finale disponible sur le dépôt Git : [Lien GitHub](#)

Les derniers profils obtenus montrent un **temps d'exécution GPU d'environ 7 s par époque**, de façon très stable :

```
==14046== Profiling result:
Type      Time(%)   Time      Calls      Avg      Min      Max      Name
GPU activities: 60.57% 1.08577s 23737 45.741us 3.3330us 211.93us matrix_dot_kernel(double const *, double const *, double*, int, int, int)
                28.76% 515.56ms 14996 34.379us 3.2810us 129.12us fused_dot_scalar_minus(double const *, double const *, double*, double, int, int, int)
                6.99% 125.33ms 11247 11.143us 1.4580us 30.678us matrix_transpose_kernel(double const *, double*, int, int)
                1.86% 33.276ms 9994 3.3290us 2.3950us 5.8330us matrix_function_kernel(double*, double*, int, bool)
                0.78% 14.007ms 9994 1.4010us 1.1450us 2.7610us matrix_sum_kernel(double*, double*, double*, int)
                0.65% 11.731ms 7498 1.5640us 1.4070us 2.4480us fused_function_hadamard(double*, double*, double*, int)
                0.39% 6.9295ms 3749 1.8480us 1.7700us 2.7610us matrix_minus_kernel(double*, double*, double*, int)
API calls: 73.03% 4.37889s 81215 53.917us 33.750us 1.5265ms cudaLaunchKernel
                16.80% 1.00740s 8746 115.18us 53.073us 1.0051ms cudaDeviceSynchronize
                5.48% 328.42ms 35 9.3834ms 80.937us 323.89ms cudaMallocManaged
                3.71% 222.73ms 71211 3.1270us 2.2400us 906.51us cudaMemcpyAsync
                0.97% 58.392ms 23737 2.4590us 1.9270us 78.854us cudaGetDevice
                0.00% 202.55us 1 202.55us 202.55us 202.55us cudaFree
                0.00% 133.39us 97 1.3750us 781ns 28.490us cuDeviceGetAttribute
                0.00% 11.979us 1 11.979us 11.979us 11.979us cuDeviceTotalMem
                0.00% 6.8230us 3 2.2740us 1.4590us 2.8640us cuDeviceGetCount
                0.00% 3.1780us 2 1.5890us 1.4590us 1.7190us cuDeviceGet
                0.00% 1.9790us 1 1.9790us 1.9790us 1.9790us cuDeviceGetName
                0.00% 1.0410us 1 1.0410us 1.0410us 1.0410us cuDeviceGetUuid
```

Figure 3 : sur 1 epoch (7 secondes)

```

==7684== Profiling result:
   Type  Time(%)   Time      Calls      Avg      Min      Max      Name
GPU activities:  57.81%  9.60300s  214906  44.684us  3.3330us  213.44us  matrix_dot_kernel(double const *, double const *, double*, int, int, int)
                30.99%  5.14843s  149960  34.332us  3.2290us  131.15us  fused_dot_scalar_minus(double const *, double const *, double*, double, int, int, int)
                7.54%  1.25190s  112470  11.130us  1.4580us  31.146us  matrix_transpose_kernel(double const *, double*, int, int)
                1.78%  296.16ms  88708  3.3380us  2.3950us  6.3540us  matrix_function_kernel(double*, double*, int, bool)
                0.75%  124.42ms  88708  1.4020us  1.1450us  3.0730us  matrix_sum_kernel(double*, double*, double*, int)
                0.71%  117.54ms  74980  1.5670us  1.4060us  3.4370us  fused_function_hadamard(double*, double*, double*, int)
                0.43%  70.799ms  37490  1.8880us  1.7700us  2.8650us  matrix_minus_kernel(double*, double*, double*, int)
API calls:      77.11%  41.6236s  767222  54.252us  33.437us  19.376ms  cudaLaunchKernel
                17.63%  9.51845s  81844  116.30us  52.656us  18.799ms  cudaDeviceSynchronize
                3.67%  1.97999s  644718  3.0710us  2.1350us  914.48us  cudaMemPrefetchAsync
                0.98%  528.81ms  214906  2.4600us  1.8750us  882.45us  cudaGetDevice
                0.61%  328.58ms  32  10.268ms  81.823us  324.58ms  cudaMallocManaged
                0.00%  189.90us  1  189.90us  189.90us  189.90us  cudaFree
                0.00%  123.96us  97  1.2770us  678ns  30.833us  cuDeviceGetAttribute
                0.00%  11.875us  1  11.875us  11.875us  11.875us  cuDeviceTotalMem
                0.00%  6.5110us  3  2.1700us  1.4580us  2.9170us  cuDeviceGetCount
                0.00%  3.3850us  2  1.6920us  1.4580us  1.9270us  cuDeviceGet
                0.00%  1.8750us  1  1.8750us  1.8750us  1.8750us  cuDeviceGetName
                0.00%  989ns  1  989ns  989ns  989ns  cuDeviceGetUuid

```

Figure 4 : Sur 10 epoch (70 secondes)

Ces mêmes profils montrent que :

- Les opérations sont aujourd'hui **MASSIVEMENT** parallélisées,
- Les transferts mémoire sont limités,
- Et le pipeline GPU est parfaitement optimisé.

Le tableau suivant résume l'évolution des performances au fil des versions :

Version du projet	Temps par epoch
Version CPU initiale	~60 s
Portage GPU classique	~120 s
GPU Mémoire unifiée	~35 s
Mémoire partagée + tuiles + autres fonctions matricielles	~32 s
Optimisation mémoire des fonctions forward et backward	~23 s
Réduction des cudaDeviceSynchronize()	~9 s
Fusion des kernels	~7 s

Cette expérience montre l'intérêt d'une **démarche incrémentale** et **structurée** pour optimiser un projet GPU :

Plutôt que de chercher à tout réécrire dès le départ, nous avons successivement optimisé chaque niveau (transferts mémoire, kernels, gestion mémoire, synchronisation) pour aboutir à un résultat **ROBUSTE ET COLOSSAL**.